WISEConv: Worklist-driven Masked Convolution for Onboard High-Speed Robotic Perception

Anonymous Authors

***Abstract*—Onboard robotic perception must sustain high update rates under tight compute and power budgets. Masking policies from event increments, temporal frame differences, and learned gates expose where computation is unnecessary, yet existing execution paths do not convert this sparsity into commensurate latency reduction. Tile skipping keeps the throughput of the dense pipeline but recomputes inactive positions at tile granularity; gather-scatter selects active positions exactly but loses throughput to feature materialization, irregular access, and writeback. We present WISEConv, a worklist-driven masked convolution operator that drives tiled convolution from an active-output worklist rather than a contiguous coordinate block, evaluating only active outputs while leaving the per-position reduction and dense tensor layout intact. Selectivity thus becomes exact without surrendering the regularity that gives the dense pipeline its throughput. A single mask-space pass fuses output-mask propagation with per-tile compaction and reuses coordinate metadata across compatible operators, keeping construction cheap and infrequent. The worklist emits tile-local segments in dense traversal order and decomposes work adaptively over the spatial and reduction dimensions, sustaining near-dense throughput without input-patch materialization or a global coordinate sort. Across various GPU platforms and perception tasks, WISEConv reduces full-model latency by 22–46% relative to the fastest competing path and per-frame energy by 28–60% over dense convolution.**

***Index Terms*—Computer architecture for robotic and automation, software, deep learning methods, sparse convolution, embedded perception.**

I. Introduction

CONVOLUTIONAL networks support a broad range of onboard robotic perception workloads, from high-rate motion and geometry estimation to detection and pose estimation [1]–[4]. Their outputs feed online navigation, planning, and manipulation, so inference latency limits how often downstream components can incorporate new observations [5]. Meeting this latency demand is difficult on board-level processors with constrained compute and power, because dense convolution recomputes every spatial position on every invocation. Prior systems avoid part of this work by deriving active masks from event-camera increments, temporal frame differences, or learned spatial gates [6]–[10]. Although these policies arise from different sensing modalities and tasks, they expose the same operator-level problem: update selected positions on dense 2D feature grids without computing inactive outputs.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates determine the input reads and output writes, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates consequently share predictable memory access and data reuse. This regularity, not merely the arithmetic count, is what a masked operator must preserve.

Existing masked-convolution systems sacrifice one part of this structure or the other. *Tile skipping* [6], [8], [11], [12] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [9], [10], [13] instead extracts active positions exactly, materializes their receptive fields, and scatters the results back to the dense grid, trading regular execution for extraction, irregular access, and writeback.

We characterize this trade-off along two axes (§II-C): *throughput* (operations per second, bounded by the dense pipeline) and *useful-compute ratio* (the fraction of executed operations that produce needed outputs). Tile skipping preserves throughput but sacrifices useful-compute ratio; gather-scatter attains a useful-compute ratio near one but sacrifices throughput and pays for extraction and writeback. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction. In our sparsest workload, the masks render 84% of the convolution work they govern unnecessary. State-of-the-art tile skipping [8] still spends half of its executed operations on unneeded outputs (a useful-compute ratio of 0.49) and cuts latency by at most 41%. Gather-scatter drives the useful-compute ratio close to one, yet sustains only 9% of tile skipping's throughput and runs 3.1× slower than dense convolution (§V-B).

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are evaluated without changing the per-position reduction, the dense tensor layout, or the weight layout. Selectivity then becomes exact by construction, pinning the useful-compute ratio near one with no input-patch materialization. A high useful-compute ratio alone does not yield latency reduction, however, since gather-scatter also attains one and still loses. What a worklist operator must additionally control is the cost of constructing the worklist and the throughput at which it is consumed.

Two challenges follow. First, *worklist-construction overhead*: constructing the worklist requires scanning the mask, and incurs cost that scales with the layer's spatial grid, not with the active count or the channel widths, while the compute the worklist enables shrinks with both the active count and the channel widths. Construction's share of latency therefore grows as activity falls and channels thin, and accumulates

Manuscript received Month DD, YYYY.

per operator when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist's order and length both affect its consumption throughput, and sparsity worsens each. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full. Both effects sharpen as activity falls.

We present WISEConv, a *Worklist-driven Masked Convolution* operator that answers both challenges by co-designing worklist construction and convolution. Construction is made cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, moving only mask and coordinate metadata, and compatible operators propagate and reuse valid coordinate metadata instead of reconstructing it. Consumption is made dense-like: construction emits contiguous per-tile segments whose order follows the dense tile traversal, giving compute tile-local spatial structure without a global sort, and compute organizes parallel work adaptively over the spatial and reduction dimensions according to device, layer shape, and activity, while each selected position retains the regular reduction over channels and kernel offsets.

This paper makes three contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, and show that tile skipping and gather-scatter each secure one at the other's expense, explaining the measured gap between required work and realized latency.
- We design WISEConv, which fuses position-level selectivity into tiled dense convolution to secure both axes at once: an exact worklist pins the useful-compute ratio near one, fused mask-space construction with cross-operator metadata reuse bounds construction cost, and tile-local segment ordering with adaptive work decomposition keeps throughput close to the dense pipeline.
- We evaluate WISEConv on four perception models spanning event increments, temporal frame differences, and learned spatial gating, across two Jetson embedded platforms and discrete laptop and desktop GPUs. WISEConv reduces full-model latency relative to the fastest baseline by 22–39% on the Jetson platforms and 34–46% on the discrete GPUs, and cuts per-frame energy over dense convolution by 28–60% on the Jetson platforms.

II. Background and Motivation

A. Latency-Critical Onboard Perception

Robotic perception runs inside an online sensing-to-action loop [14], [15]. Optical flow supplies motion estimates and depth supplies obstacle geometry for navigation [1], [2], segmentation supplies scene state for planning [3], and object pose and grasp proposals guide manipulation [4], [5]. In each case the inference output feeds a downstream component whose next decision depends on the freshness of that estimate [16], [17]. Because freshness is set by latency, a lower perception latency is always preferable: where a task imposes a latency budget, a lower latency makes that budget easier to meet; where none is fixed, it still lets the controller react to the environment sooner [18], [19]. We therefore target latency minimization rather than a preset deadline. Offloading to remote compute could add capacity but is bounded by communication delay and connectivity [20], so we close the loop onboard under a limited power and compute budget [21], minimizing end-to-end perception latency on the target device.

One line of work is targeted to lower this latency by computing only the active region of each frame, delivered as a spatial mask. The mask derives from event-camera increments [6], [7], temporal residuals between video frames [8], [12], [22], or learned input-dependent gates [9], [10], [13], [23]. These sources differ in sensing modality and policy, but all expose the same spatial sparsity on a dense 2D feature grid for the convolution operator to exploit.

B. Masked Convolution

Masked convolution keeps the dense tensors and arithmetic of a dense convolution and restricts only which output coordinates it evaluates. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate p has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus fixes which positions of $\mathbf{y}$ the operator updates, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$. The active ratio $\alpha = \|M_{out}\|_0 / |\mathcal{G}|$, the active fraction of the grid, is the intrinsic sparsity the policy exposes.

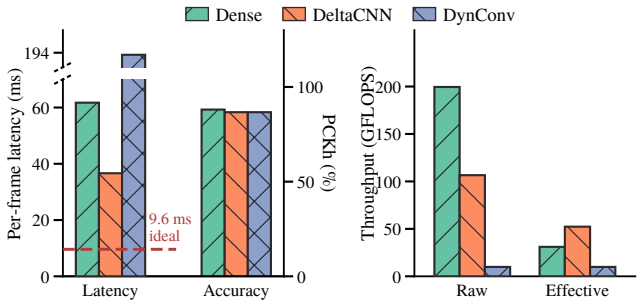
An execution path realizes this mask over a compute set $\mathcal{C}$ bounded by $\{p : M_{out}[p] = 1\} \subseteq \mathcal{C} \subseteq \mathcal{G}$; for each $p \in \mathcal{C}$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \, \mathbf{x}[\mathcal{N}_i(p)], \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. The path must compute every active coordinate to avoid dropping a requested output but may also compute inactive ones; dense convolution is the extreme $\mathcal{C} = \mathcal{G}$. The policy alone fixes the accuracy and the required work, while the choice of $\mathcal{C}$ is determined by the path.

C. The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [6], [11], [12] inherits the dense pipeline's tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [8] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, cutting wasted work while degrading throughput. *Gather-scatter* [9],



(a) Latency and accuracy (b) Throughput

Fig. 1. Statistics of DynConv [10], a pose estimator with learned spatial gating, on the MPII human-pose dataset [24] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff}, aggregated across the model's convolution layers. The dense useful-compute ratio implies the active ratio set by DynConv's spatial gating ($\alpha \approx 15.6\%$).

[13] instead extracts exactly the active coordinates, computes, and scatters them back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [10] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

We measure each path by two quantities. Its useful-compute ratio is the fraction of computed coordinates that are active,

$$\eta = \frac{\|M_{out}\|_0}{|\mathcal{C}|} \leq 1. \tag{3}$$

Its throughput is the rate at which it processes issued FLOPs,

$$T = \frac{2\,|\mathcal{C}|\,k^2 C_{in} C_{out}}{t}, \tag{4}$$

where t is the per-frame latency and the leading factor of 2 counts each multiply-accumulate as two FLOPs. For the required $2\,\|M_{out}\|_0\,k^2 C_{in} C_{out}$ FLOPs, latency is set by the **effective throughput** $T_{\text{eff}} = \eta T$, the rate at which a path computes needed outputs.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [8]) reaches 36.6 ms, still 3.8× the floor, and gather-scatter, the path DynConv is built around, runs at 194 ms, far slower than dense convolution itself. That is due to their low effective throughput from different causes. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 0.49; gather-scatter lifts the useful-compute ratio to one, yet extraction and data movement collapse its throughput to 5% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput.

D. Relation to 3D Sparse Convolution

SpConv v2 and TorchSparse++ target point clouds that are sparse by representation, building coordinate maps over irregular voxel coordinates [25], [26]. Our setting instead keeps dense 2D tensors under a changing spatial mask, where the challenge is to skip inactive outputs without turning a regular image-grid pipeline into an irregular one. We therefore treat 3D sparse convolution as related infrastructure rather than a primary baseline.

III. WISEConv Design

A. Overview

WISEConv turns an upstream activity mask into the coordinate stream of a tiled convolution. A layer receives a dense feature tensor $\mathbf{x}$ and its input mask M_{in}. It produces the output values $\mathbf{y}$ requested by the mask policy, the propagated output mask M_{out}, and a worklist $\mathbf{q}$ of active output coordinates. The operator separates this work into two stages (Figure 2):

1) **Construction stage.** A mask-space kernel evaluates Eq. (1), writes M_{out}, and compacts its active coordinates into $\mathbf{q}$. It scans the output grid in spatial tiles, so each emitted segment retains the local traversal order of one tile.
2) **Compute stage.** A tiled convolution kernel reads its output coordinates from $\mathbf{q}$ instead of enumerating a contiguous output region. For those coordinates, it preserves the regular reduction over input channels and kernel offsets used by dense convolution.

This separation removes coordinate discovery from the convolution kernel's critical path and establishes spatial structure before feature data are accessed. Construction touches only masks and coordinates; compute retains dense feature and weight tensors and executes one output for every worklist entry. The following sections describe tiled construction and worklist-driven compute.

B. Tiled Worklist Construction

Construction must determine the active output set before convolution without moving feature data or imposing a device-wide ordering pass. We partition the output grid into contiguous construction tiles $T_1, \ldots, T_N$, each with a fixed local traversal. For every position $p \in T_n$, the stage evaluates Eq. (1) and writes the result to $M_{out}[p]$. It then selects the active positions within that tile:

$$\mathcal{I}_n = \{j : M_{out}[T_n[j]] = 1\}, \tag{5}$$

and emits their linear output coordinates in local traversal order:

$$\mathbf{q}_n = (T_n[j])_{j \in \mathcal{I}_n}. \tag{6}$$

Tiles execute in parallel and reserve disjoint segments of the final worklist. If P denotes their reservation order, the resulting array is

$$\mathbf{q} = [\mathbf{q}_{P_1}; \ldots; \mathbf{q}_{P_N}], \qquad |\mathbf{q}| = \|M_{out}\|_0. \tag{7}$$

Parallel scheduling may change the order of these segments, so WISEConv does not require or provide a global raster

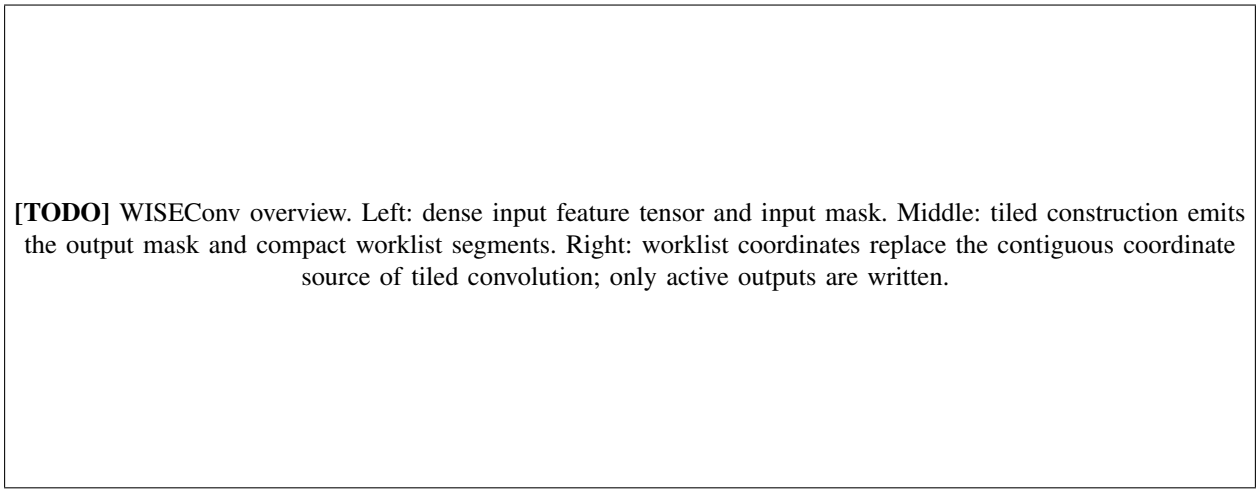


Fig. 2. The WISEConv two-stage pipeline. Construction discovers active output coordinates and compacts each spatial tile into a worklist segment. Compute uses the resulting coordinates to drive a tiled convolution while retaining its regular channel and kernel reduction.

Algorithm 1 Tiled Worklist Construction

Require: Construction tiles $T_1, \ldots, T_N$, input mask M_{in}
Ensure: Worklist $\mathbf{q}$, output mask M_{out}

for each tile T_n **in parallel do**
  for each position $p \in T_n$ **do**
    $M_{out}[p] \leftarrow$ Eq. (1)
  end for
  $\mathbf{q}_n \leftarrow (T_n[j])_{j:M_{out}[T_n[j]]=1}$
  $b_n \leftarrow \text{reserve}(|\mathbf{q}_n|)$
  $\mathbf{q}[b_n : b_n + |\mathbf{q}_n|] \leftarrow \mathbf{q}_n$
end for

order. Its guarantee is tile-local: every $\mathbf{q}_n$ contains coordinates from one contiguous output region in that tile's traversal order. Generating this order during compaction avoids a subsequent sort over the active set.

Algorithm 1 summarizes the stage. The fused pass visits $H_{out}W_{out}$ output positions and examines at most k^2 input-mask entries per position. Its $O(H_{out}W_{out}k^2)$ work is independent of C_{in} and C_{out}. We include construction in operator latency and measure its cost directly in Section V-D.

C. Worklist-Driven Convolution

Implicit GEMM views convolution as a matrix product whose rows correspond to output coordinates, columns to output channels, and reduction dimension to input channels and kernel offsets. A dense kernel derives each row coordinate from a contiguous output tile. WISEConv instead loads those coordinates from $\mathbf{q}$; the column and reduction dimensions remain regular.

The compute stage partitions the worklist into blocks of B_M coordinates, the output channels into blocks of B_N, and the reduction into steps of B_K. Each compute tile first loads a coordinate block $\mathbf{q}'$. For every reduction step, it derives input addresses from $\mathbf{q}'$, loads the corresponding input and weight tiles, and accumulates their products. It then writes the result to the dense output tensor at the same coordinates (Algorithm 2). The worklist schedules exactly the support of M_{out}.

Algorithm 2 Worklist-Driven Convolution

Require: Worklist $\mathbf{q}$, input $\mathbf{x}$, weight $\mathbf{w}$
Ensure: Output $\mathbf{y}$ at coordinates in $\mathbf{q}$

for each output tile (b_m, b_n) **in parallel do**
  $\mathbf{q}' \leftarrow \mathbf{q}[(b_m{-}1)B_M : b_m B_M]$
  $\mathbf{a} \leftarrow \mathbf{0}_{B_M \times B_N}$
  for each reduction tile r of width B_K **do**
    $\mathbf{x}' \leftarrow$ input tile addressed by $(\mathbf{q}', r)$
    $\mathbf{w}' \leftarrow$ weight tile addressed by (b_n, r)
    $\mathbf{a} \leftarrow \mathbf{a} + \mathbf{x}'\mathbf{w}'$
  end for
  $\mathbf{y}[\mathbf{q}', b_n] \leftarrow \mathbf{a}$
end for

The input tile in Algorithm 2 is loaded directly from $\mathbf{x}$ into the kernel's working storage; it is not materialized as a separate global matrix. Construction order makes these loads less irregular than an arbitrary active-coordinate list. Entries within each $\mathbf{q}_n$ address nearby output positions and therefore nearby, frequently overlapping input neighborhoods. A compute tile inherits this locality when its coordinate block lies within a construction segment. It may also cross segment boundaries, which affects locality but not correctness. This limited guarantee lets compute exploit available spatial structure without requiring globally sorted coordinates.

IV. Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates

once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section III-B without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split-K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split-K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight preparation complete before the timed sequence described in Section V-A.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

V. Evaluation

A. Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

1) Platforms: We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

2) Models, Tasks, and Mask Sources: Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [27], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [28]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [6], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on `indoor_flying1` and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [29] on MOT16 [30], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [8] supplies the temporal mask mechanism by propagating thresholded activation deltas between successive frames.
- **Human pose estimation.** We use the DynConv pose model [10] on MPII [24], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table I summarizes their models and evaluation scales. We use official weights for all four models [10], [27], [29]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

TABLE I
Evaluation workloads.

	FireFlowNet [27]	**YOLOv8n/m** [29]	**DynConv Pose** [10]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [6]	DeltaCNN [8]	DynConv [10]
Dataset	MVSEC [28]	MOT16 [30]	MPII [24]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

3) Execution Baselines: We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [31].
- **Tile skipping** follows the tile-sparse execution established by SBNet, Ev-Conv, and DeltaCNN [6], [8], [11].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [10] and an implementation equivalent to MMCV `MaskedConv2d` for FireFlowNet and YOLO [32].

4) Metrics and Methodology: Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on

the model's compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work}, useful-compute ratio η, convolution throughput, and the construction fraction of operator latency. Because layers differ in cost, an unweighted average of layer activities misstates the work a mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\text{work}} = \frac{\sum_l \rho^l W^l_{\text{dense}}}{\sum_l W^l_{\text{dense}}}, \qquad \rho^l = \frac{\|M^l\|_0}{H^l_{out} W^l_{out}}, \tag{8}$$

where W^l_{dense} is the dense MAC count of layer l. Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every requested output.

B. Full-Model Latency

C. Efficiency Analysis

D. Construction Overhead

E. Task Accuracy

ACKNOWLEDGMENTS

REFERENCES

[1] D. Wofk, F. Ma, T. Yang, S. Karaman, and V. Sze, "Fastdepth: Fast monocular depth estimation on embedded systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6101–6108.

[2] R. J. Bouwmeester, F. Paredes-Vallés, and G. C. H. E. de Croon, "Nanoflownet: Real-time dense optical flow on a nano quadcopter," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 1996–2003.

[3] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, "Bisenet: Bilateral segmentation network for real-time semantic segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 334–349.

[4] C. Wang, D. Xu, Y. Zhu, R. M. Martin, C. Lu, L. Fei-Fei, and S. Savarese, "Densefusion: 6d object pose estimation by iterative dense fusion," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 3343–3352.

[5] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 13 438–13 444.

[6] S. Durvasula, Y. Guan, and N. Vijaykumar, "Ev-conv: Fast CNN inference on event camera inputs for high-speed robot perception," *IEEE Robotics Autom. Lett.*, vol. 8, no. 6, pp. 3174–3181, 2023.

[7] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, "Event-based asynchronous sparse convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 415–431.

[8] M. Parger, C. Tang, C. D. Twigg, C. Keskin, R. Wang, and M. Steinberger, "Deltacnn: End-to-end CNN inference of sparse frame differences in videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 12 487–12 496.

[9] F. Li, G. Li, X. He, and J. Cheng, "Dynamic dual gating neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 5310–5319.

[10] T. Verelst and T. Tuytelaars, "Dynamic convolutions: Exploiting spatial sparsity for faster inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2320–2329.

[11] M. Ren, A. Pokrovsky, B. Yang, and R. Urtasun, "Sbnet: Sparse blocks network for fast inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 8711–8720.

[12] A. Habibian, D. Abati, T. S. Cohen, and B. E. Bejnordi, "Skip-convolutions for efficient video processing," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 2695–2704.

[13] Z. Xie, Z. Zhang, X. Zhu, G. Huang, and S. Lin, "Spatially adaptive inference with stochastic feature sampling and interpolation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 531–548.

[14] L. Tai and M. Liu, "Deep-learning in mobile robotics - from perception to control systems: A survey on why and why not," *CoRR*, vol. abs/1612.07139, 2016. [Online]. Available: http://arxiv.org/abs/1612.07139

[15] A. Waga, S. Benhlima, A. Bekri, J. Abdouni, and F. Z. Saber, "A survey on autonomous navigation for mobile robots: From traditional techniques to deep learning and large language models," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 37, no. 7, 2025. [Online]. Available: https://doi.org/10.1007/s44443-025-00216-x

[16] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "PAMPC: perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 2018, pp. 1–8. [Online]. Available: https://doi.org/10.1109/IROS.2018.8593739

[17] D. Falanga, S. Kim, and D. Scaramuzza, "How fast is too fast? the role of perception latency in high-speed sense and avoid," *IEEE Robotics Autom. Lett.*, vol. 4, no. 2, pp. 1884–1891, 2019. [Online]. Available: https://doi.org/10.1109/LRA.2019.2898117

[18] D. Falanga, K. Kleber, and D. Scaramuzza, "Dynamic obstacle avoidance for quadrotors with event cameras," *Sci. Robotics*, vol. 5, no. 40, p. 9712, 2020. [Online]. Available: https://doi.org/10.1126/scirobotics.aaz9712

[19] D. Hanover, A. Loquercio, L. Bauersfeld, A. Romero, R. Penicka, Y. Song, G. Cioffi, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing: A survey," *IEEE Trans. Robotics*, vol. 40, pp. 3044–3067, 2024. [Online]. Available: https://doi.org/10.1109/TRO.2024.3400838

[20] B. Kehoe, S. Patil, P. Abbeel, and K. Goldberg, "A survey of research on cloud robotics and automation," *IEEE Trans Autom. Sci. Eng.*, vol. 12, no. 2, pp. 398–409, 2015. [Online]. Available: https://doi.org/10.1109/TASE.2014.2376492

[21] D. Falanga, E. Mueggler, M. Faessler, and D. Scaramuzza, "Aggressive quadrotor flight through narrow gaps with onboard sensing and computing using active vision," in *2017 IEEE International Conference on Robotics and Automation, ICRA 2017, Singapore, Singapore, May 29 - June 3, 2017*. IEEE, 2017, pp. 5774–5781. [Online]. Available: https://doi.org/10.1109/ICRA.2017.7989679

[22] L. Cavigelli, P. Degen, and L. Benini, "Cbinfer: Change-based inference for convolutional neural networks on video data," in *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*, 2017, pp. 1–8.

[23] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. P. Vetrov, and R. Salakhutdinov, "Spatially adaptive computation time for residual networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1790–1799.

[24] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, "2d human pose estimation: New benchmark and state of the art analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 3686–3693.

[25] Y. Yan, Y. Mao, and B. Li, "SECOND: sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.

[26] H. Tang, S. Yang, Z. Liu, K. Hong, Z. Yu, X. Li, G. Dai, Y. Wang, and S. Han, "Torchsparse++: Efficient training and inference framework for sparse convolution on gpus," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2023, pp. 225–239.

[27] F. Paredes-Vallés and G. C. H. E. de Croon, "Back to event basics: Self-supervised learning of image reconstruction for event cameras via photometric constancy," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 3445–3454.

[28] A. Z. Zhu, D. Thakur, T. Özaslan, B. Pfrommer, V. Kumar, and K. Daniilidis, "The multi vehicle stereo event camera dataset: An event camera dataset for 3d perception," *CoRR*, vol. abs/1801.10202, 2018.

[29] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[30] A. Milan, L. Leal-Taixé, I. D. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *CoRR*, vol. abs/1603.00831, 2016.

[31] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," *CoRR*, vol. abs/1410.0759, 2014.

[32] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and

D. Lin, "Mmdetection: Open mmlab detection toolbox and benchmark," *CoRR*, vol. abs/1906.07155, 2019.